CIUDAD 3D INTERACTIVA

Descripción del proyecto, funcionalidades, controles, limitaciones e instrucciones de ejecución

Implementación basada en Java, LWJGL, GLFW y OpenGL 3.3

1. Descripción general

El proyecto consiste en una aplicación interactiva que representa una pequeña ciudad tridimensional y permite conducir un vehículo por sus calles. La escena se genera mediante geometría sencilla (principalmente cajas y cubos) y se renderiza con OpenGL. El usuario puede alternar entre cámaras, cambiar la iluminación, encender o apagar los faros, observar un minimapa y completar una secuencia de entregas.

2. Diseño y estructura de la ciudad 3D

La ciudad se define a partir de una matriz bidimensional llamada MAPA (11x11) donde cada celda representa una parcela cuadrada de 10 unidades por lado (CELDA = 10) y las filas y columnas alternan calles y manzanas, formando una cuadrícula urbana. Los valores de la matriz tienen el siguiente significado:

Valor	Significado	Representación
0	Calle o espacio transitable	Superficie vial y marcas de calle.
1	Parcela edificada	Edificio construido con geometría de cajas.
2	Parque	Área verde con árboles y un banco.

3. Organización del código

Archivo / clase	Responsabilidad principal
clase1.java	Inicializa GLFW/OpenGL, crea la ventana y los shaders, define el mapa, dibuja la ciudad y gestiona la cámara base.
clase2.java	Añade el vehículo, conducción, aceleración, frenado, giro, colisiones y cámara de seguimiento.
clase3.java	Añade modo día/noche, farolas, faros del vehículo y cálculo de iluminación en el fragment shader.
clase4.java	Añade entregas, cronómetro, destinos, decoración urbana, semáforos, pasos peatonales y minimapa.
HUD.java	Dibuja una interfaz 2D superpuesta con estado de iluminación, cámara, controles, entregas y velocidad.
VehiculoAutonomo	Dibuja vehiculos que simulant un trafico autonomo en la ciudad

4. Funcionalidades implementadas

4.1. Renderizado y cámara

- Ventana inicial de 1100 × 800 píxeles, creada con GLFW.
- Solicitud de contexto OpenGL 3.3 Core y activación de la prueba de profundidad.
- Ciclo principal que procesa eventos, calcula el tiempo entre cuadros, actualiza el estado y dibuja la escena.
- Cámara de seguimiento situada detrás y por encima del vehículo, y cámara aérea/general para observar la ciudad.
- Minimapa cuadrado en la esquina superior derecha, renderizado mediante un segundo pase de dibujo con vista superior.

4.2. Vehículo y conducción

- Vehículo construido con cajas: carrocería, cabina, cuatro ruedas, faros delanteros y luces traseras.
- Aceleración y desaceleración dependientes del tiempo entre cuadros; velocidad máxima hacia delante de 16 unidades/s y reversa limitada a −6 unidades/s.
- Resistencia de rodadura y freno más fuerte al mantener la barra espaciadora.

- Giro mediante A y D, calculado según la velocidad del vehículo.
- Detección de colisiones con parcelas de edificios/parques y con los límites del mapa. Si la posición propuesta no es válida, el vehículo se detiene.

4.3. Iluminación

- Modo día/noche que modifica la luz ambiental y la contribución direccional.
- Farolas con postes y bombillas cálidas. Sus posiciones se calculan desde una lista de coordenadas de fila/columna y un indicador de orientación.
- Faros delanteros con un haz calculado en el shader, orientado según el ángulo del vehículo.
- Uso de uniforms GLSL para enviar al shader el estado de noche, faros, posición y dirección del vehículo, y posiciones de las farolas.

4.4. Entregas y elementos urbanos

- Tres destinos definidos por coordenadas en DESTINOS.
- Una marca dorada indica el destino activo; en la vista principal aparece también una baliza que oscila y gira.
- Una entrega se completa cuando el vehículo está a menos de 3 unidades del destino y su velocidad absoluta es inferior a 1 unidad/s.
- Cronómetro que mide el tiempo de recorrido y mensaje de victoria al completar las tres entregas.
- Semáforos decorativos en intersecciones seleccionadas: el ciclo visual usa rojo, verde y amarillo en fases temporizadas.
- Pasos peatonales, ventanas, árboles y bancos como elementos decorativos.

4.5. HUD

HUD.java genera una imagen transparente con Java AWT, dibuja paneles redondeados y texto, la sube a una textura OpenGL y la representa sobre un cuadrilátero que ocupa la pantalla. Muestra el estado día/noche, faros, modo de cámara, controles, progreso de entregas y velocidad aproximada en km/h.

5. Controles del usuario

Tecla	Acción
W	Acelerar hacia delante.
S	Reducir la velocidad hacia delante y continuar en reversa si se mantiene.
A	Girar hacia la izquierda.
D	Girar hacia la derecha.
Barra espaciadora	Frenar con mayor intensidad mientras se mantiene pulsada.
C	Alternar entre cámara de seguimiento y cámara aérea/general.
Flecha Derecha	Gira la camara aerea hacia la derecha.
Flecha Izquierda	Gira la camara aerea hacia la izquierda.
N	Alternar entre iluminación diurna y nocturna.
F	Encender o apagar los faros del vehículo.
M	Mostrar u ocultar el minimapa.
R	Reiniciar posición, orientación, velocidad y progreso de entregas.
Esc	Cerrar la aplicación.

6. Limitaciones conocidas

- El shader implementa iluminación local, pero no sombras.
- No se observa una gestión avanzada de materiales, reflejos o niebla.
- Los semáforos son decorativos: cambian de color por tiempo, pero no coordinan tráfico ni regulan la conducción del vehículo.

- No hay peatones, rutas de navegación ni simulación de reglas de tránsito.
- No se incluyen animaciones complejas ni texturas artísticas externas.
- El HUD crea una imagen del tamaño actual al construirse. HUD se inicializa una vez; no se aprecia un método de redimensionamiento de su imagen/textura si cambia el tamaño de la ventana.
- El código no muestra pruebas automatizadas, guardado de progreso ni configuración de niveles.

7. Requisitos e instrucciones de ejecución

7.1. Requisitos

- JDK compatible con text blocks de Java; se recomienda JDK 17 o posterior.
- LWJGL 3 con los módulos usados por el código: lwjgl, lwjgl-glfw y lwjgl-opengl.
- Bibliotecas nativas de LWJGL que correspondan al sistema operativo y arquitectura del equipo.
- Controlador gráfico compatible con OpenGL 3.3 Core.
- IDE Java (por ejemplo, IntelliJ IDEA, Eclipse o VS Code con extensiones Java) o un sistema de construcción como Maven/Gradle.

7.2. Ejecución desde un IDE

1. Abra o cree un proyecto Java y configure el JDK.
2. Añada los módulos LWJGL indicados y los archivos nativos correspondientes a su sistema operativo.
3. Copie los cinco archivos Java al paquete com.graphics.
4. Verifique que el IDE resuelva los imports org.lwjgl.* y las llamadas de OpenGL.
5. Ejecute el método main de clase4 para iniciar la versión completa del proyecto.

También se puede ejecutar clase1 o clase2 para probar etapas anteriores, y clase3 para probar la etapa de iluminación. Para la experiencia completa, el punto de entrada recomendado es com.graphics.clase4.

8. Recursos externos, bibliotecas y atribución

8.1. Dependencias externas identificables

Recurso	Uso en el proyecto
LWJGL	Enlace entre Java y las APIs nativas de gráficos y ventana.
GLFW (incluido mediante LWJGL)	Creación de ventana, eventos de teclado, ciclo de eventos y sincronización de cuadros.
OpenGL (a través de LWJGL)	Renderizado de geometría, buffers, texturas, shaders y estados gráficos.
Java AWT / BufferedImage / Graphics2D	Generación de la imagen 2D y textos del HUD.
GLSL 3.30	Lenguaje de los shaders de vértices y fragmentos incluidos como cadenas de texto dentro del código Java.